Ps 05

Erik An

<2026-04-27 Mon>

Contents

1 Problem 44 **3**

1.1 Task 3

1.2 Solution 3

1.2.1 (a) 3

1.2.2 (b) 4

1.2.3 (c) 5

1.2.4 (d) 6

2 Problem 45 **7**

2.1 Task 7

2.2 Solution 7

2.2.1 (a) 7

2.2.2 (b) 8

2.2.3 (c) 8

2.2.4 (d) 9

3 Problem 46 **10**

3.1 Task 10

3.2 Solution 11

3.2.1 (a) 11
3.2.2 (b) 11
3.2.3 (c) 11
3.2.4 (d) 13

4 Problem 47 **13**
4.1 Task 13
4.2 Solution 14

5 Problem 48 **15**
5.1 Task 15
5.2 Solution 15

6 Problem 49 **18**
6.1 Task 18
6.2 Solution 18

7 Problem 50 **21**
7.1 Task 21
7.2 Solution 21

8 Problem 51 **23**
8.1 Task 23
8.2 Solution 24
8.2.1 (a) 24
8.2.2 (b) 24
8.2.3 (c) 26

9 Errors/Typos **26**

1 Problem 44

1.1 Task

Searching an unsorted array 1 This problem examines an algorithm for searching for a value x in an unsorted array A consisting of n elements called `random_search`. Consider the following randomized strategy: pick a random index i into A. If $A[i] = x$, then terminate; otherwise, continue the search by picking a new random index into A. Continue picking random indices into A until you find an index j such that $A[j] = x$ or until every element of A has been checked. This strategy may examine a given element more than once, because it picks from the whole set of indices each time. In the following we assume that all possible permutations of the input array are equally likely.

(a) Implement a procedure `random_search` to realise the strategy above. Be sure that your algorithm terminates when all indices into A have been picked.

(b) Suppose that there is exactly one index i such that $A[i] = x$. What is the expected number of indices into A that must be picked before x is found and `random_search` terminates?

(c) Suppose now that there are $k > 1$ indices i such that $A[i] = x$. What is the expected number of indices into A that must be picked before x is found and random search terminates? Your answer should be a function of n and k.

(d) Suppose that there are no indices i such that $A[i] = x$. What is the expected number of indices into A that must be picked before all elements of A have been checked and random search terminates?

1.2 Solution

1.2.1 (a)

```
using Random

let
    function random_search(A, x)
        seen = falses(length(A))
        count = 0

        while !all(seen)
            count += 1
            i = rand(eachindex(A))
            A[i] == x && return (i, count)
            seen[i] = true
        end

        return (nothing, count)
    end
```

```
    arr = [1, 2, 3, 4, 5, 7, 7, 8, 9, 0]
    idx, _ = random_search(arr, 7)
    println("found at index $idx")
end
```

1.2.2 (b)

```
using Random

let
    function random_search(A, x)
        seen = falses(length(A))
        count = 0

        while !all(seen)
            count += 1
            i = rand(eachindex(A))
            A[i] == x && return (i, count)
            seen[i] = true
        end

        return (nothing, count)
    end

    mean_picks(A, x; trials=1_000_000) = sum(random_search(A, x)[2] for _ in 1:trials) / trials

    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
    println("E(T)  $(mean_picks(arr, 7))")
end
```

Let $n = |A|$. Each pick is i.i.d. Bernoulli$(1/n)$: independence from sampling with replacement, identical distribution from uniform index choice. Let T be the number of picks until the first success.

When $x \in A$, the unique successful index lies among the n indices, so the algorithm returns before the `seen` guard fires; the bounded T equals the unbounded geometric T almost surely.

$$
\begin{aligned}
P(T = k) &= (1-p)^{k-1}p, \quad k = 1, 2, \ldots \\
E(T) &= \sum_{k=1}^{\infty} k\,(1-p)^{k-1}p \\
&= p \sum_{k=1}^{\infty} k(1-p)^{k-1} \\
&= p \cdot \frac{1}{(1-(1-p))^2} \qquad \left(\sum_{k=1}^{\infty} k x^{k-1} = \frac{1}{(1-x)^2},\ |x| < 1 \right) \\
&= \frac{1}{p} = n.
\end{aligned}
$$

Sanity check: $n = 10$, predicted $E(T) = 10$, empirical 9.993.

1.2.3 (c)

```
using Random

let
    function random_search(A, x)
        seen = falses(length(A))
        count = 0

        while !all(seen)
            count += 1
            i = rand(eachindex(A))
            A[i] == x && return (i, count)
            seen[i] = true
        end

        return (nothing, count)
    end

    mean_picks(A, x; trials=1_000_000) = sum(random_search(A, x)[2] for _ in 1:trials) / trials

    arr = [1, 2, 3, 4, 5, 7, 7, 8, 9, 0]
    println("E(T)  $(mean_picks(arr, 7))")
end
```

By the same argument as (b): each pick is i.i.d. Bernoulli(p), but now $p = k/n$, since k of the n indices yield $A[i] = x$. The `seen` guard does not fire (at least one successful index exists, so the loop returns first), so T is geometric. Hence

$$E(T) = \frac{1}{p} = \frac{n}{k}.$$

Sanity check: $n = 10$, $k = 2$, predicted $E(T) = 5$, empirical 4.998.

1.2.4 (d)

```
using Random

let
    function random_search(A, x)
        seen = falses(length(A))
        count = 0

        while !all(seen)
            count += 1
            i = rand(eachindex(A))
            A[i] == x && return (i, count)
            seen[i] = true
        end

        return (nothing, count)
    end

    mean_picks(A, x; trials=1_000_000) = sum(random_search(A, x)[2] for _ in 1:trials) / trials

    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
    println("E(T)  $(mean_picks(arr, 10))")
end
```

Define T_i be a number of picks made while you have exactly $i - 1$ distinct indices seen, until the new i th element was picked. Hence, $T = T_1 + T_2 + \ldots + T_n$.

Each T_i is geometric, at the moment $i-1$ was picked, the probability of picking new is $p_i = \frac{(n-i+1)}{n}$.

$$
\begin{aligned}
E(T) &= E\left(\sum_{i=1}^{n} T_i\right) \\
&= \sum_{i=1}^{n} E(T_i) \qquad \text{(linearity of expectation)} \\
&= \sum_{i=1}^{n} \frac{1}{p_i} \qquad \text{(by (b), since } T_i \sim \text{Geometric}(p_i)\text{)} \\
&= \sum_{i=1}^{n} \frac{n}{n-i+1} \\
&= n \sum_{j=1}^{n} \frac{1}{j} \qquad (j = n-i+1) \\
&= nH_n.
\end{aligned}
$$

Sanity check: $n = 10$, predicted $E(T) \approx 29.29$, empirical 29.29.

2 Problem 45

2.1 Task

Searching an unsorted array 2 This problem examines the search algorithm `deterministic_search`. This algorithm searches A for x in order, considering $A[1], A[2], \ldots A[n]$, until either it finds $A[i] = x$ or it reaches the end of the array. In the following we again assume that all possible permutations of the input array are equally likely.

(a) Implement a procedure `deterministic_search` to realise the strategy above.

(b) Suppose that there is exactly one index i such that $A[i] = x$. What is the average-case running time of `deterministic_search`? What is its worst-case running time?

(c) Suppose that there are $k > 1$ indices i such that $A[i] = x$. What is the average-case running time of `deterministic_search`? What is the worst-case running time of `deterministic_search`? Your answer should be a function of n and k.

(d) Suppose that there are no indices i such that $A[i] = x$. What is the average-case running time of `deterministic_search`? What is its worst-case running time?

2.2 Solution

2.2.1 (a)

```
using Random
```

```
let
    function deterministic_search(A, x)
        for i in eachindex(A)
            A[i] == x && return i
        end
        return nothing
    end

    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
    idx = deterministic_search(arr, 7)
    println("found at index $idx")
end
```

2.2.2 (b)

Assume uniform distribution of all possible permutations of an array, with $p_i = \frac{1}{n}$, where $n = |A|$.

By definition of the algorithm, it does i comparisons to reach position i. Hence,

$$
\begin{aligned}
T_i &= i \\
T_{\text{avg}} &= \sum_{i=1}^{n} p_i \cdot T_i \\
&= \sum_{i=1}^{n} \frac{i}{n} \\
&= \frac{1}{n} \cdot \sum_{i=1}^{n} i \\
&= \frac{1}{n} \cdot \frac{n(n+1)}{2} \\
&= \frac{n+1}{2}
\end{aligned}
$$

Worst case running time would be traversing through every element of the array, hence $\Theta(n)$.

2.2.3 (c)

```
using Random

let
    function deterministic_search(A, x)
        for i in eachindex(A)
```

```
            A[i] == x && return i
        end
        return nothing
    end

    mean_picks(A, x; trials=1_000_000) = sum(deterministic_search(shuffle(A), x) for _ in 1:tria

    arr = [1, 2, 3, 4, 5, 7, 7, 8, 9, 0]
    println("T_avg  $(mean_picks(arr, 7))")
end
```

$$
\begin{aligned}
P(\text{first match at } j) &= \frac{\binom{n-j}{k-1}}{\binom{n}{k}} \\
&= \frac{\text{arrangements with first x at position j}}{\text{total arrangement of x's positions}} \\
T_{\text{avg}} &= \sum_{j=1}^{n-k+1} j \cdot P(\text{first match at } j) \\
&= \sum_{j=1}^{n-k+1} j \cdot \frac{\binom{n-j}{k-1}}{\binom{n}{k}}
\end{aligned}
$$

Note, this expression can be simplified, I just do not want to do it.

Worst case running time would be traversing through every non x element of the array, hence $\Theta(n-k+1)$.

$$
\begin{aligned}
T_{\text{avg}} &= \sum_{j=1}^{10-2+1} j \cdot \frac{\binom{10-j}{2-1}}{\binom{10}{2}} \\
&= \frac{11}{3} \\
&\approx 3.666
\end{aligned}
$$

Sanity check: $n = 10, k = 2$, predicted $T_{\text{avg}} \approx 3.666$, empirical 3.667.

2.2.4 (d)

```
using Random
```

```
let
    function deterministic_search(A, x)
        count = 0

        for i in eachindex(A)
            count += 1
            if A[i] == x
                break
            end
        end
        return count
    end

    mean_picks(A, x; trials=1_000_000) = sum(deterministic_search(shuffle(A), x) for _ in 1:tria

    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
    println("T_avg  $(mean_picks(arr, 10))")
end
```

As there is no $i : A[i] = x$, the algorithm would traverse whole array, resulting in $\Theta(n)$, where $n = |A|$.

Sanity check: $n = 10, k = 0$, predicted $P(\text{first match at } j) = n = 10$, empirical 10.0.

3 Problem 46

3.1 Task

Searching an unsorted array 3 This problem examines the search algorithm `scramble_search`. It first randomly permutes the input array and then runs the algorithm `deterministic_search` on the resulting permuted array. In the following we also assume that all possible permutations of the input array are equally likely.

(a) Implement a procedure `scramble_search` to realise the strategy above. Use your implementation of `deterministic_search` from Exercise 45.

(b) Suppose that there is exactly one index i such that $A[i] = x$. What is the expected number of indices into A that must be picked before x is found and `scramble_search` terminates?

(c) Suppose now that there are $k > 1$ indices i such that $A[i] = x$. What is the expected number of indices into A that must be picked before x is found and `scramble_search` terminates? Your answer should be a function of n and k.

(d) Suppose that there are no indices i such that $A[i] = x$. What is the expected number of indices into A that must be picked before all elements of A have been checked and `scramble_search` terminates?

Which of the three searching algorithms `random_search`, `deterministic_search`, or `scramble_search` would you use and why?

3.2 Solution

3.2.1 (a)

```
using Random

let
    function deterministic_search(A, x)
        for i in eachindex(A)
            if A[i] == x
                return i
            end
        end
        return nothing
    end

    function scramble_search(A, x)
        return deterministic_search(shuffle(A), x)
    end

    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
    idx = scramble_search(arr, 7)
    println("found at index $idx")
end
```

3.2.2 (b)

Let J be the index of x after shuffling. Since all permutations are equiprobable, $J \equiv \text{Uniform}\{1, \ldots, n\}$, and $T = J$.

$$E(T) = \sum_{j=1}^{n} j \cdot \frac{1}{n} = \frac{n+1}{2}$$

3.2.3 (c)

```
using Random

let
    function deterministic_search(A, x)
```

```
        for i in eachindex(A)
            A[i] == x && return i
        end
        return nothing
    end

    scramble_search(A, x) = deterministic_search(shuffle(A), x)

    mean_picks(A, x; trials=1_000_000) = sum(scramble_search(A, x) for _ in 1:trials) / trials

    arr = [1, 2, 3, 4, 5, 7, 7, 8, 9, 0]
    println("E(T)  $(mean_picks(arr, 7))")
end
```

Let J be the position of the first match after shuffling. With k matches placed uniformly at random among n positions:

$$P(J = j) = \frac{\binom{n-j}{k-1}}{\binom{n}{k}}, \quad 1 \le j \le n - k + 1$$
$$E(J) = \sum_{j=1}^{n-k+1} j \cdot \frac{\binom{n-j}{k-1}}{\binom{n}{k}}$$

Substituting $i = n - j$ and applying the hockey-stick identity twice, using:

$$i\binom{i}{k-1} = k\binom{i}{k} + (k-1)\binom{i}{k-1}$$

$$\begin{aligned}\sum_{j=1}^{n-k+1} j\binom{n-j}{k-1} &= n\binom{n}{k} - k\binom{n}{k+1} - (k-1)\binom{n}{k} \\ &= (n-k+1)\binom{n}{k} - k\binom{n}{k+1} \\ &= \frac{n+1}{k+1}\binom{n}{k}\end{aligned}$$

Hence

$$E(T) = \frac{n+1}{k+1}$$

Sanity: $n = 10, k = 2 \Rightarrow \frac{11}{3} \approx 3.667$, matching the shuffle simulation from Problem 44(c).

3.2.4 (d)

```
using Random

let
    function deterministic_search(A, x)
        for i in eachindex(A)
            A[i] == x && return i
        end
        return length(A)
    end

    scramble_search(A, x) = deterministic_search(shuffle(A), x)

    mean_picks(A, x; trials=1_000_000) = sum(scramble_search(A, x) for _ in 1:trials) / trials

    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
    println("E(T)  $(mean_picks(arr, 10))")
end
```

With no i such that $A[i] = x$, every permutation forces the algorithm to examine all n elements before terminating. Hence $T = n$ deterministically and

$$E(T) = n$$

Sanity: $n = 10 \Rightarrow E(T) = 10$, empirical 10.0 (zero variance, as expected).

4 Problem 47

4.1 Task

> Scheduling In the Linux system all processes get a base priority value between 0 and 139. The process scheduler computes a real priority value from the process's base priority value. Lower priority values are better than higher priority values, i.e. from two processes the one with higher priority is the one with lower real priority value. Assume that the process scheduler always picks the next process among those with lowest real priority value. Assume further that every process has in addition to its two priority values the following additional information of the specified type: process id (integer), process name (string), user id (integer), group id (integer), registers (array(integer) of size 20), process flags (array(boolean) of size 20), floating point registers (array(double) of size 16).
>
> Implement a heap data structure that can be used for the priority queue as described above.

4.2 Solution

```
let
    mutable struct Node
        base_priority::UInt8
        real_priority::UInt8
        process_id::Int64
        process_name::String
        user_id::Int64
        group_id::Int64
        registers::NTuple{20, Int64}
        process_flags::NTuple{20, Bool}
        f_registers::NTuple{16, Float64}
    end

    struct MinHeap
        data::Vector{Node}
        MinHeap() = new(Node[])
    end

    #-----------------------------------------

    parent(i) = i ÷ 2
    left(i)   = 2i
    right(i)  = 2i + 1

    #-----------------------------------------

    function Base.push!(h::MinHeap, n::Node)
        d = h.data
        push!(d, n)
        i = length(d)
        while i > 1 && d[parent(i)].real_priority > d[i].real_priority
            d[parent(i)], d[i] = d[i], d[parent(i)]
            i = parent(i)
        end
        return h
    end

    function Base.pop!(h::MinHeap)
        d = h.data
        isempty(d) && return nothing
        result = d[1]
        d[1] = d[end]
        pop!(d)
        i = 1
        while left(i) <= length(d)
```

```
            j = left(i)
            if right(i) <= length(d) && d[right(i)].real_priority < d[j].real_priority
                j = right(i)
            end
            d[i].real_priority <= d[j].real_priority && break
            d[i], d[j] = d[j], d[i]
            i = j
        end
        return result
    end

    Base.peek(h::MinHeap) = h.data[1]

    #-----------------------------------------

    make_node(p) = Node(0x00, UInt8(p), 0, "", 0, 0, ntuple(_->0,20), ntuple(_->false,20), ntupl

    ps = [50, 10, 90, 30, 70, 20, 40, 80, 60, 5, 100, 1]
    h = MinHeap()
    foreach(p -> push!(h, make_node(p)), ps)
    @assert [Int(pop!(h).real_priority) for _ in ps] == sort(ps)
    println("ok")

end
```

5 Problem 48

5.1 Task

Implement a priority queue data structure for the process scheduler using the implemented heap of Exercise 47 and handles.

5.2 Solution

```
let
    mutable struct Handle
        idx::Int
    end

    mutable struct Node
        base_priority::UInt8
        real_priority::UInt8
        process_id::Int64
        process_name::String
```

```
    user_id::Int64
    group_id::Int64
    registers::NTuple{20, Int64}
    process_flags::NTuple{20, Bool}
    f_registers::NTuple{16, Float64}
end

struct MinHeap
    data::Vector{Node}
    handles::Vector{Handle}
    MinHeap() = new(Node[], Handle[])
end

#-----------------------------------------

parent(i) = i ÷ 2
left(i)   = 2i
right(i)  = 2i + 1

#-----------------------------------------

function swap!(h::MinHeap, i::Int, j::Int)
    h.data[i], h.data[j] = h.data[j], h.data[i]
    h.handles[i], h.handles[j] = h.handles[j], h.handles[i]
    h.handles[i].idx = i
    h.handles[j].idx = j
end

function siftup!(h::MinHeap, i::Int)
    d = h.data
    while i > 1 && d[parent(i)].real_priority > d[i].real_priority
        swap!(h, i, parent(i))
        i = parent(i)
    end
end

function siftdown!(h::MinHeap, i::Int)
    d = h.data
    while left(i) <= length(d)
        j = left(i)
        if right(i) <= length(d) && d[right(i)].real_priority < d[j].real_priority
            j = right(i)
        end
        d[i].real_priority <= d[j].real_priority && break
        swap!(h, i, j)
        i = j
    end
end
```

```
function Base.push!(h::MinHeap, n::Node)
    push!(h.data, n)
    i = length(h.data)
    handle = Handle(i)
    push!(h.handles, handle)
    siftup!(h, i)
    return handle
end

function Base.pop!(h::MinHeap)
   isempty(h.data) && return nothing
   result = h.data[1]
   popped_handle = h.handles[1]
   swap!(h, 1, length(h.data))
   pop!(h.data); pop!(h.handles)
   popped_handle.idx = 0
   siftdown!(h, 1)
   return result
end

Base.peek(h::MinHeap) = h.data[1]

function decrease_key(h::MinHeap, handle::Handle, rp)
    handle.idx == 0 && return
    @assert rp <= h.data[handle.idx].real_priority
    h.data[handle.idx].real_priority = UInt8(rp)
    siftup!(h, handle.idx)
end

function increase_key(h::MinHeap, handle::Handle, rp)
    handle.idx == 0 && return
    @assert rp >= h.data[handle.idx].real_priority
    h.data[handle.idx].real_priority = UInt8(rp)
    siftdown!(h, handle.idx)
end

#-----------------------------------------

make_node(p) = Node(0x00, UInt8(p), 0, "", 0, 0, ntuple(_->0,20), ntuple(_->false,20), ntupl

ps = [50, 10, 90, 30, 70, 20, 40, 80, 60, 5, 100, 1]
h = MinHeap()
foreach(p -> push!(h, make_node(p)), ps)
@assert [Int(pop!(h).real_priority) for _ in ps] == sort(ps)
h2 = MinHeap()
hs = [push!(h2, make_node(p)) for p in [50, 80, 30]]
decrease_key(h2, hs[2], 5)
```

```
    @assert pop!(h2).real_priority == 5
    println("ok")
end
```

6 Problem 49

6.1 Task

Implement a simple scheduler that performs a loop, such that every iteration performs the following algorithm:

(a) Pick and remove the highest priority process P from the priority queue.

(b) Decrease the real priority value of every process still in the queue by 1.

(c) With a 10% chance discard the process P (it has terminated), with a 90% chance re-insert the process P into the queue with a new real priority value of 10 times its base priority value plus 1.

(d) With a 10% chance randomly create a new process with random base priority in the range $[1, 138]$ and put it in the queue with real priority value equal to its base priority value.

Then populate the priority queue with a process called "kernel" with process id 0, user id 0, group id 0, base priority value 1, the rest random, a process called "idle" with process id 1, user id 0, group id 0, base priority value 139, the rest random, and three other completely random processes. Then perform 1000 iterations of your process scheduler and print the process that was chosen in every iteration.

6.2 Solution

```
using Random

let
    mutable struct Handle
        idx::Int
    end

    mutable struct Node
        base_priority::Int
        real_priority::Int
        process_id::Int
        process_name::String
        user_id::Int
        group_id::Int
        registers::NTuple{20, Int64}
        process_flags::NTuple{20, Bool}
        f_registers::NTuple{16, Float64}
```

```
end

struct MinHeap
    data::Vector{Node}
    handles::Vector{Handle}
    MinHeap() = new(Node[], Handle[])
end

#-----------------------------------------

parent(i) = i ÷ 2
left(i)   = 2i
right(i)  = 2i + 1

#-----------------------------------------

function swap!(h::MinHeap, i::Int, j::Int)
    h.data[i], h.data[j] = h.data[j], h.data[i]
    h.handles[i], h.handles[j] = h.handles[j], h.handles[i]
    h.handles[i].idx = i
    h.handles[j].idx = j
end

function siftup!(h::MinHeap, i::Int)
    d = h.data
    while i > 1 && d[parent(i)].real_priority > d[i].real_priority
        swap!(h, i, parent(i))
        i = parent(i)
    end
end

function siftdown!(h::MinHeap, i::Int)
    d = h.data
    while left(i) <= length(d)
        j = left(i)
        if right(i) <= length(d) && d[right(i)].real_priority < d[j].real_priority
            j = right(i)
        end
        d[i].real_priority <= d[j].real_priority && break
        swap!(h, i, j)
        i = j
    end
end

function Base.push!(h::MinHeap, n::Node)
    push!(h.data, n)
    i = length(h.data)
    handle = Handle(i)
```

```
    push!(h.handles, handle)
    siftup!(h, i)
    return handle
end

function Base.pop!(h::MinHeap)
   isempty(h.data) && return nothing
   result = h.data[1]
   popped_handle = h.handles[1]
   swap!(h, 1, length(h.data))
   pop!(h.data); pop!(h.handles)
   popped_handle.idx = 0
   siftdown!(h, 1)
   return result
end

Base.peek(h::MinHeap) = h.data[1]

function decrease_key(h::MinHeap, handle::Handle, rp)
    handle.idx == 0 && return
    @assert rp <= h.data[handle.idx].real_priority
    h.data[handle.idx].real_priority = Int(rp)
    siftup!(h, handle.idx)
end

function increase_key(h::MinHeap, handle::Handle, rp)
    handle.idx == 0 && return
    @assert rp >= h.data[handle.idx].real_priority
    h.data[handle.idx].real_priority = Int(rp)
    siftdown!(h, handle.idx)
end

#-----------------------------------------

function random_node(; pid=rand(Int), uid=rand(Int), gid=rand(Int), base=rand(1:138), name=r
    Node(
        base,
        base,
        pid, name, uid, gid,
        ntuple(_ -> rand(Int),    20),
        ntuple(_ -> rand(Bool),   20),
        ntuple(_ -> rand(),       16),
    )
end

function step!(h::MinHeap)
    isempty(h.data) && return nothing
    hp = pop!(h)
```

```
        for n in h.data
            n.real_priority -= 1
        end
        if rand() < 0.9
            hp.real_priority = hp.base_priority * 10 + 1
            push!(h, hp)
        end
        if rand() < 0.1
            push!(h, random_node())
        end
        return hp
    end

    function scheduler(h::MinHeap, n::Int=1000)
        for _ in 1:n
            p = step!(h)
            (p === nothing) || println(p.process_name, " (pid=", p.process_id, ")")
        end
    end

    h = MinHeap()
    push!(h, random_node(pid=0, uid=0, gid=0, base=1,   name="kernel"))
    push!(h, random_node(pid=1, uid=0, gid=0, base=139, name="idle"))
    for _ in 1:3
        push!(h, random_node())
    end
    scheduler(h, 1000)
end
```

7 Problem 50

7.1 Task

> When RANDOMIZED-QUICKSORT runs, how many calls are made to the random-number generator RANDOM in the worst case? How about in the best case? Give your answer in terms of Θ notation.

7.2 Solution

RANDOMIZED-PARTITION calls RANDOM exactly once each time it runs, so counting RANDOM calls is the same as counting partition calls. Let $f(n)$ be the number of partition calls on an array of size n, with $f(0) = f(1) = 0$ (no partition is called on an empty or singleton subarray). If a partition produces sides of sizes k and $n - 1 - k$, then

$$f(n) = 1 + f(k) + f(n-1-k).$$

To bound $f(n)$ I look at the recursion tree. Let $I = f(n)$ be the number of internal nodes (partition calls) and $L = I + 1$ the number of leaves (a property of any binary tree where every internal node has two children). Each partition fixes exactly one element as its pivot, so out of the n original elements, $n - I$ of them end up alone in size-1 leaves. The rest of the leaves are size-0:

$$L_1 = n - I, \qquad L_0 = (I+1) - L_1 = 2I + 1 - n.$$

Since $L_0 \geq 0$, we get $I \geq \lceil (n-1)/2 \rceil$. So no matter what splits happen, $f(n) = \Omega(n)$.

- Worst case

  This is when partitions are as unbalanced as possible: the pivot is always the smallest or largest element, so one side has size 0 and the other has size $n - 1$. Then

  $$f(n) = 1 + f(n-1), \quad f(1) = 0 \implies f(n) = n - 1 = \Theta(n).$$

- Best case

  To minimize $f(n)$ I want to minimize the number of partition calls, which by the bound above means making L_0 as small as possible. Splitting into sizes 1 and $n - 2$ at every step gives $L_0 = 0$ (when n is odd) and saturates the bound:

  $$f(n) = 1 + f(1) + f(n-2) = 1 + f(n-2) = \lceil (n-1)/2 \rceil = \Theta(n).$$

  Note: in CLRS the "best case" usually means the median split, because that is what minimizes the **running time**. For just the RANDOM count, $(1, n-2)$ splits are actually fewer calls than median splits, but both are $\Theta(n)$, so the asymptotic answer is the same either way.

So both the best and worst case make $\Theta(n)$ calls to RANDOM. This is different from the number of comparisons, which is $\Theta(n \log n)$ in the best case and $\Theta(n^2)$ in the worst case. The reason RANDOM is the same in both cases: every partition costs exactly one coin flip, and the lower bound $I \geq \lceil (n-1)/2 \rceil$ already forces $\Theta(n)$ partition calls regardless of how balanced the splits are.

I checked this with a small simulation in Julia:

```
using Random
let
    function randomized_partition!(A, lo, hi)
        p = rand(lo:hi)
        A[p], A[hi] = A[hi], A[p]
        pivot = A[hi]
        i = lo - 1
        @inbounds for j in lo:hi-1
```

```
            if A[j]  pivot
                i += 1
                A[i], A[j] = A[j], A[i]
            end
        end
        A[i+1], A[hi] = A[hi], A[i+1]
        return i + 1
    end
    function randomized_quicksort!(A, lo=1, hi=length(A))
        if lo < hi
            q = randomized_partition!(A, lo, hi)
            randomized_quicksort!(A, lo, q - 1)
            randomized_quicksort!(A, q + 1, hi)
        end
        return A
    end
    randomized_quicksort(A) = randomized_quicksort!(copy(A))

    # Counting RANDOM calls in the three split shapes
    worst(n)        = n  1 ? 0 : 1 + worst(n - 1)
    best(n)         = n  1 ? 0 : cld(n - 1, 2)
    function median_split(n)
        n  1 && return 0
        k = (n - 1) ÷ 2
        1 + median_split(k) + median_split(n - 1 - k)
    end

    A = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
    println(randomized_quicksort(A))
    for n in (10, 100, 1000)
        println("n=$n  worst=$(worst(n))  best=$(best(n))  median=$(median_split(n))")
    end
end
```

The numbers confirm that all three cases grow linearly in n.

8 Problem 51

8.1 Task

Professors Howard, Fine, and Howard have proposed a deceptively simple sorting algorithm, named stooge sort in their honor.

1: procedure STOOGE-SORT(A,p,r) 2: if $A[p] > A[r]$ then 3: exchange $A[p]$ with $A[r]$ 4: end if 5: if $p+1 < r$ then 6: $k = \lfloor (r-p+1)/3 \rfloor$ 7: STOOGE-SORT(A, p, r - k) / *first*

two-thirds 8: STOOGE-SORT(A, p + k, r) / last two-thirds 9: STOOGE-SORT(A, p, r - k) // first two-thirds again 10: end if 11: end procedure

(a) Argue that the call STOOGE-SORT(A, 1, n) correctly sorts the array $A[1:n]$.

(b) Give a recurrence for the worst-case running time of STOOGE-SORT and a tight asymptotic (Θ notation) bound on the worst-case running time.

(c) Compare the worst-case running time of STOOGE-SORT with that of insertion sort, merge sort, heapsort, and quicksort. Do the professors deserve tenure?

8.2 Solution

8.2.1 (a)

Induction on $n = r - p + 1$.

Base. $n \leq 2$: lines 2–3 perform the only nontrivial work and produce a sorted pair (or singleton); the recursion guard $p + 1 < r$ blocks further calls.

Step. $n \geq 3$, so $k = \lfloor n/3 \rfloor \geq 1$ and each recursive call has size $n - k = \lceil 2n/3 \rceil < n$. Let $L = A[p..r-k]$ and $R = A[p+k..r]$, both of size $n - k$, with $|L \cap R| = n - 2k \geq 0$ since $k \leq n/3$.

By IH all three recursive calls sort their subarrays. Trace:

1. After call 1 (L sorted): the k largest entries of L sit at positions $[r - 2k + 1,\ r - k]$. This range lies inside R, since $p + k \leq r - 2k + 1 \iff 3k \leq n$, which holds.
2. The k globally largest entries of $A[p..r]$ are either originally in L — now in its top-k tail $\subset R$ — or in $A[r - k + 1..r] \subset R$. Either way, after call 1 they all sit inside R.
3. After call 2 (R sorted): the top k of R — equal to the top k of A by step 2 — occupy $A[r - k + 1..r]$ in sorted order.
4. After call 3 (L sorted): L holds the $n - k$ smallest entries of A in sorted order. Concatenated with the sorted $A[r - k + 1..r]$, $A[p..r]$ is sorted.

8.2.2 (b)

Outside recursion the work is $\Theta(1)$ (one comparison, optional swap). Three recursive calls on size $\lceil 2n/3 \rceil$:

$$T(n) = 3\,T\left(\left\lceil \tfrac{2n}{3} \right\rceil\right) + \Theta(1).$$

Master theorem with $a = 3$, $b = 3/2$: $\log_b a = \log_{3/2} 3 = \dfrac{\ln 3}{\ln 3 - \ln 2} \approx 2.7095$. Since $f(n) = \Theta(1) = O(n^{\log_b a - \varepsilon})$ for small ε, case 1 applies:

$$T(n) = \Theta\Big(n^{\log_{3/2} 3}\Big) \approx \Theta(n^{2.71}).$$

```
using Random

let
    ops = Ref(0)

    function stooge_sort!(A, p=1, r=length(A))
        ops[] += 1
        if A[p] > A[r]
            A[p], A[r] = A[r], A[p]
        end
        if p + 1 < r
            k = (r - p + 1) ÷ 3
            stooge_sort!(A, p, r - k)
            stooge_sort!(A, p + k, r)
            stooge_sort!(A, p, r - k)
        end
        return A
    end

    Random.seed!(0)
    for _ in 1:500
        n = rand(1:40)
        A = rand(-100:100, n)
        @assert stooge_sort!(copy(A)) == sort(A)
    end
    println("correctness ok")

     = log(3) / log(1.5)
    for n in (50, 100, 500, 1000)
        ops[] = 0
        stooge_sort!(rand(n))
        slope = log(ops[]) / log(n)
        println("n=$n  ops=$(ops[])  log(ops)/log(n) = $(round(slope, digits=4))")
    end
    println(" = log_{3/2}(3)  $(round(, digits=4))")
end
```

Sanity check: empirical log–log slopes oscillate around $\alpha \approx 2.71$; the wobble comes from floor/ceiling effects in $k = \lfloor n/3 \rfloor$, not from a violated bound.

8.2.3 (c)

Algorithm	Worst case
Insertion sort	$\Theta(n^2)$
Merge sort	$\Theta(n \log n)$
Heapsort	$\Theta(n \log n)$
Quicksort	$\Theta(n^2)$
Stooge sort	$\Theta(n^{\log_{3/2} 3}) \approx \Theta(n^{2.71})$

Stooge sort is asymptotically worse than every algorithm in the comparison, including the quadratic ones. Tenure denied.

9 Errors/Typos

- Problem 44: "called $\text{random}_{\text{search}}$]." → stray "]" (likely orphaned bracket from "[random search]" or footnote marker)
- Problem 44(c): "ramdom search" → "random search"
- Problem 45: "This problem examine the search algorithm" → "This problem examines the search algorithm"
- Problem 47: "all processes gets a base priority value" → "all processes get a base priority value"
- Problem 51: `end if` on line 7 is misplaced — the three recursive calls (lines 8–10) should be inside the `if p + 1 < r` block, otherwise k is undefined when $p + 1 \geq r$. The `end if` should be on line 11 (after the recursive calls), and the procedure's `end procedure` on line 12.